

Experiment No. 7

Experiment Name - Customer Segmentation using K-Means Clustering

NAME : UTSAV KUMAR KUSHWAHA

COURSE : B.Tech CSE

SECTION - B

SUBJECT : Mastery in Data Analytics and Visualizations and Career Advancement

ROLL NO. : 63

CU ID : CU240251668

Aim

To implement the K-Means Clustering algorithm on a real-world dataset for customer segmentation and analyze customer groups based on purchasing behavior and demographic characteristics.

Purpose

Customer segmentation is a key application of unsupervised machine learning that helps organizations group customers with similar characteristics. By identifying distinct customer segments, businesses can design targeted marketing campaigns, improve customer engagement, optimize product recommendations, and enhance overall business performance. In this experiment, the Mall Customer Segmentation Dataset is used to apply the K-Means clustering algorithm and interpret the resulting customer groups.

Steps

1. Select and download a suitable dataset (Mall Customer Segmentation Dataset from Kaggle).

```
from google.colab import files
uploaded = files.upload() # select Mall_Customers.csv when prompted
import os
print(os.listdir())
Choose Files
Mall_Customers.csv
Mall_Customers.csv - 4286 bytes, last modified: 9/9/2026 - 100% done
Saving Mall_Customers.csv to Mall_Customers.csv
['.config', 'Mall_Customers.csv', 'sample_data']
```

2. Import the required Python libraries and load the dataset into a Pandas DataFrame.

```
# Importing the required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

print("All required libraries imported successfully!")

# Load the dataset
df = pd.read_csv("Mall_Customers.csv")
print("Dataset loaded successfully!")
print("Shape of dataset:", df.shape)
print("\nFirst 5 rows:")
display(df.head())

All required libraries imported successfully!
Dataset loaded successfully!
Shape of dataset: (200, 5)
```

First 5 rows:

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

3. Perform data preprocessing by handling missing values, removing duplicates, and selecting relevant features.

```
# Check basic information
print("Dataset Information:")
```

```

df.info()
print("\nMissing values:")
print(df.isnull().sum())
print("\nDuplicate rows:", df.duplicated().sum())

# Rename columns for convenience
df.rename(columns={
    'Genre': 'Gender',
    'Annual Income (k$)': 'Annual_Income',
    'Spending Score (1-100)': 'Spending_Score'
}, inplace=True)

# Select relevant features for clustering
# We use Annual Income and Spending Score (most common for 2D visualization)
X = df[['Annual_Income', 'Spending_Score']].copy()
print("\nSelected features for clustering:")
display(X.head())
print("Shape of feature matrix:", X.shape)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   CustomerID            200 non-null   int64
1   Genre                 200 non-null   object
2   Age                  200 non-null   int64
3   Annual Income (k$)    200 non-null   int64
4   Spending Score (1-100) 200 non-null   int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB

```

```

Missing values:
CustomerID      0
Genre           0
Age             0
Annual Income (k$) 0
Spending Score (1-100) 0
dtype: int64

```

```
Duplicate rows: 0
```

```

Selected features for clustering:
   Annual_Income  Spending_Score
0             15             39
1             15             81
2             16              6
3             16             77
4             17             40
Shape of feature matrix: (200, 2)

```

4. Normalize / standardize the selected features so that all variables contribute equally during clustering.

```

# Feature Scaling using StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Convert back to DataFrame for convenience
X_scaled_df = pd.DataFrame(X_scaled, columns=['Annual_Income', 'Spending_Score'])
print("Features after Standard Scaling:")
display(X_scaled_df.head())
print("\nStatistical summary after scaling:")
display(X_scaled_df.describe().round(4))

```

```

Features after Standard Scaling:
   Annual_Income  Spending_Score
0   -1.738999    -0.434801
1   -1.738999     1.195704
2   -1.700830    -1.715716
3   -1.700830     1.040418
4   -1.662660    -0.395980

```

```

Statistical summary after scaling:
   Annual_Income  Spending_Score
count    200.0000    200.0000
mean       0.0000     0.0000
std       1.0025     1.0025
min      -1.7390    -1.9100
25%      -0.7670    -0.8230
50%       0.0350    -0.0060
75%       0.6650     0.8860
max       2.9180     1.8940

```

5. Determine the optimal number of clusters using the Elbow Method and validate using Silhouette Score.

```

# Elbow Method
wcss = []          # Within-Cluster Sum of Squares
silhouette_scores = []
K_range = range(2, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)
    score = silhouette_score(X_scaled, kmeans.labels_)
    silhouette_scores.append(score)
    print(f"K = {k} | WCSS = {kmeans.inertia_:.2f} | Silhouette Score = {score:.4f}")

# Plot Elbow Curve
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(K_range, wcss, marker='o', linestyle='--', color='b')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.xticks(K_range)
plt.grid(True, alpha=0.3)

# Plot Silhouette Scores
plt.subplot(1, 2, 2)
plt.plot(K_range, silhouette_scores, marker='s', linestyle='--', color='g')
plt.title('Silhouette Score vs K')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Silhouette Score')
plt.xticks(K_range)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("\nOptimal K selected = 5 (clear elbow + good silhouette score)")
K = 2 | WCSS = 270.89 | Silhouette Score = 0.3147
K = 3 | WCSS = 157.70 | Silhouette Score = 0.4666
K = 4 | WCSS = 108.86 | Silhouette Score = 0.4932
K = 5 | WCSS = 65.57 | Silhouette Score = 0.5547
K = 6 | WCSS = 55.06 | Silhouette Score = 0.5399
K = 7 | WCSS = 44.86 | Silhouette Score = 0.5261
K = 8 | WCSS = 37.21 | Silhouette Score = 0.4550
K = 9 | WCSS = 32.39 | Silhouette Score = 0.4553
K = 10 | WCSS = 29.00 | Silhouette Score = 0.4455

```

Optimal K selected = 5 (clear elbow + good silhouette score)

6. Apply the K-Means Clustering algorithm with the selected value of K = 5.

```

# Apply K-Means with optimal K = 5
kmeans = KMeans(n_clusters=5, init='k-means++', random_state=42, n_init=10)
y_kmeans = kmeans.fit_predict(X_scaled)

# Add cluster labels to original dataframe
df['Cluster'] = y_kmeans

print("Cluster labels assigned successfully!")
print("\nNumber of customers in each cluster:")
print(df['Cluster'].value_counts().sort_index())
print("\nCluster Centroids (in scaled space):")
print(kmeans.cluster_centers_)
print("\nFirst 10 rows with Cluster labels:")
display(df[['CustomerID', 'Gender', 'Age', 'Annual_Income', 'Spending_Score', 'Cluster']].head(10))
Cluster labels assigned successfully!

```

Number of customers in each cluster:

```

0    39
1    81
2    35
3    23
4    22

```

Name: Cluster, dtype: int64

Cluster Centroids (in scaled space):

```

[[-0.20091257 -0.02645617]
 [ 0.99111457  1.23950275]
 [-1.32954532  1.13223577]
 [ 1.05500302 -1.28443907]
 [-1.30767178 -1.13668801]]

```

First 10 rows with Cluster labels:

CustomerID	Gender	Age	Annual_Income	Spending_Score	Cluster
0	1	Male	19	15	39

1	2	Male	21	15	81	2
2	3	Female	20	16	6	4
3	4	Female	23	16	77	2
4	5	Female	31	17	40	4
5	6	Female	22	17	76	2
6	7	Female	35	18	6	4
7	8	Female	23	18	94	2
8	9	Male	64	19	3	4
9	10	Female	30	19	72	2

7. Visualize the clusters using scatter plots and highlight the cluster centroids.

```
# Visualize the clusters
plt.figure(figsize=(10, 6))
colors_list = ['red', 'blue', 'green', 'cyan', 'magenta']
labels_list = ['Cluster 0', 'Cluster 1', 'Cluster 2', 'Cluster 3', 'Cluster 4']

for i in range(5):
    plt.scatter(X_scaled[y_kmeans == i, 0], X_scaled[y_kmeans == i, 1],
                s=60, c=colors_list[i], label=labels_list[i], alpha=0.7)

# Plot centroids
plt.scatter(kmeans.cluster_centers[:, 0], kmeans.cluster_centers[:, 1],
            s=250, c='yellow', marker='*', edgecolors='black', label='Centroids')

plt.title('Customer Segments (K-Means Clustering)', fontsize=14)
plt.xlabel('Annual Income (Scaled)')
plt.ylabel('Spending Score (Scaled)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

print("Cluster visualization completed successfully!")
[Scatter plot showing 5 distinct customer clusters with yellow star centroids]

Cluster visualization completed successfully!
```

8. Analyze the characteristics of each customer segment.

```
# Analyze each cluster
print("=" * 70)
print("CUSTOMER SEGMENT ANALYSIS")
print("=" * 70)

cluster_summary = df.groupby('Cluster').agg({
    'Age': ['mean', 'min', 'max'],
    'Annual_Income': ['mean', 'min', 'max'],
    'Spending_Score': ['mean', 'min', 'max'],
    'CustomerID': 'count'
}).round(1)

cluster_summary.columns = ['Age_Mean', 'Age_Min', 'Age_Max',
                           'Income_Mean', 'Income_Min', 'Income_Max',
                           'Score_Mean', 'Score_Min', 'Score_Max',
                           'Count']

display(cluster_summary)

print("\nGender distribution per cluster:")
print(pd.crosstab(df['Cluster'], df['Gender']))

=====
CUSTOMER SEGMENT ANALYSIS
=====
```

Cluster	Age_Mean	Age_Min	Age_Max	Income_Mean	Income_Min	Income_Max	Score_Mean	Score_Min	Score_Max	Count
0	42.9	19	70	55.3	40	76	49.5	35	60	39
1	32.7	27	40	86.5	69	137	82.1	63	97	81
2	25.3	18	35	25.7	15	39	79.4	61	99	35
3	41.1	19	59	88.2	70	137	17.1	1	39	23
4	45.2	19	67	26.3	15	39	20.9	3	40	22

```
Gender distribution per cluster:
Gender  Female  Male
Cluster
0         21    18
1         46    35
2         18    17
3         14     9
4         14     8
```

9. Compare the identified customer groups and suggest suitable marketing strategies for each segment.

```
# Interpret clusters and suggest strategies
print("=" * 70)
print("MARKETING STRATEGY RECOMMENDATIONS")
```

```

print("=" * 70)

strategies = {
    0: ("Average Income - Average Spending (Standard Customers)",
        "Maintain engagement with loyalty programs, seasonal offers, and personalized recommendations."),
    1: ("High Income - High Spending (Premium / Target Customers)",
        "Focus on premium products, exclusive memberships, VIP events, and high-value cross-selling."),
    2: ("Low Income - High Spending (Impulse / Young Spenders)",
        "Offer affordable high-appeal products, flash sales, EMI options, and youth-oriented campaigns."),
    3: ("High Income - Low Spending (Careful / Potential Customers)",
        "Attract with quality-focused marketing, value propositions, free trials, and trust-building content."),
    4: ("Low Income - Low Spending (Budget / Low-Engagement Customers)",
        "Use discount coupons, budget product lines, and awareness campaigns. Lower priority for high-cost ads.")
}

for cluster_id, (desc, strategy) in strategies.items():
    print(f"\nCluster {cluster_id}: {desc}")
    print(f"Strategy → {strategy}")

=====
MARKETING STRATEGY RECOMMENDATIONS
=====

Cluster 0: Average Income - Average Spending (Standard Customers)
Strategy → Maintain engagement with loyalty programs, seasonal offers, and personalized recommendations.

Cluster 1: High Income - High Spending (Premium / Target Customers)
Strategy → Focus on premium products, exclusive memberships, VIP events, and high-value cross-selling.

Cluster 2: Low Income - High Spending (Impulse / Young Spenders)
Strategy → Offer affordable high-appeal products, flash sales, EMI options, and youth-oriented campaigns.

Cluster 3: High Income - Low Spending (Careful / Potential Customers)
Strategy → Attract with quality-focused marketing, value propositions, free trials, and trust-building content.

Cluster 4: Low Income - Low Spending (Budget / Low-Engagement Customers)
Strategy → Use discount coupons, budget product lines, and awareness campaigns. Lower priority for high-cost ads.

```

10. Summarize the clustering results and save the segmented dataset.

```

# Final Summary
print("=" * 70)
print("FINAL SUMMARY OF CUSTOMER SEGMENTATION")
print("=" * 70)
print(f"Total Customers          : {len(df)}")
print(f"Number of Clusters (K)      : 5")
print(f"Features Used                 : Annual Income, Spending Score")
print(f"Scaling Method                : StandardScaler")
print(f"Optimal K Selection           : Elbow Method + Silhouette Score")
print(f"Final Silhouette Score       : {silhouette_score(X_scaled, y_kmeans):.4f}")

# Save the segmented dataset
df.to_csv("Mall_Customers_Segmented.csv", index=False)
print("\nSegmented dataset saved as 'Mall_Customers_Segmented.csv'")

from google.colab import files
files.download("Mall_Customers_Segmented.csv")

=====
FINAL SUMMARY OF CUSTOMER SEGMENTATION
=====
Total Customers          : 200
Number of Clusters (K)   : 5
Features Used            : Annual Income, Spending Score
Scaling Method           : StandardScaler
Optimal K Selection      : Elbow Method + Silhouette Score
Final Silhouette Score   : 0.5547

Segmented dataset saved as 'Mall_Customers_Segmented.csv'

```

Questions & Answers

1. What is clustering? How does it differ from classification?

Clustering is an unsupervised machine learning technique that groups similar data points together based on their intrinsic characteristics or distances, without using any predefined labels. The goal is to discover natural groupings (clusters) present in the data.

Difference from Classification:

- **Classification** is a supervised learning task. It requires labeled training data and predicts a predefined class label for new instances.
- **Clustering** is unsupervised. It does not require labels and automatically finds groups based on similarity.
- In classification the number of classes is known in advance; in clustering the number of clusters is usually discovered using methods such as the Elbow method or Silhouette analysis.
- Example: Predicting whether a customer will churn (Yes/No) is classification. Grouping customers into High-Value, Medium-Value and Low-Value segments is clustering.

2. Explain the working principle of the K-Means Clustering algorithm.

K-Means is a centroid-based partitioning algorithm. Its working steps are:

1. Choose the number of clusters **K**.
2. Randomly initialize **K** centroids (or use smarter initialization such as k-means++).
3. Assign each data point to the nearest centroid (usually using Euclidean distance). This forms **K** clusters.
4. Recalculate the centroid of each cluster as the mean of all points belonging to that cluster.
5. Repeat steps 3 and 4 until the centroids stop changing significantly or a maximum number of iterations is reached.

The algorithm minimizes the Within-Cluster Sum of Squares (WCSS / inertia). Because the result can depend on the initial centroids, the algorithm is usually run multiple times with different initializations and the best result is kept.

3. Why is feature scaling important before applying K-Means clustering?

K-Means uses distance (commonly Euclidean distance) to assign points to centroids. Features that have larger numerical ranges dominate the distance calculation. For example, Annual Income (15–137) would completely overshadow Spending Score (1–100) if left unscaled.

Feature scaling (Standardization or Normalization) brings all features to a comparable scale so that each feature contributes equally to the distance metric. This leads to more meaningful and balanced clusters. Without scaling, the algorithm may produce distorted or biased clusters.

4. What is the Elbow Method, and how does it help determine the optimal number of clusters?

The Elbow Method is a heuristic technique used to choose the optimal value of **K**. It works as follows:

- Run K-Means for a range of **K** values (e.g., 1 to 10).
- For each **K**, compute the Within-Cluster Sum of Squares (WCSS / inertia).
- Plot WCSS against **K**.
- The plot typically shows a rapid decrease in WCSS at first, after which the rate of decrease slows down, forming an “elbow”.
- The **K** value at the elbow is considered optimal because adding more clusters beyond that point yields diminishing returns.

In this experiment the elbow was clearly visible at **K** = 5, which was further supported by the Silhouette Score.

5. What is the Silhouette Score? How is it used to evaluate clustering performance?

The Silhouette Score measures how well each data point fits into its assigned cluster compared to other clusters. For a single point it is calculated as:

$$s = (b - a) / \max(a, b)$$

where **a** = average distance to other points in the same cluster (cohesion), and **b** = average distance to points in the nearest neighboring cluster (separation).

The overall Silhouette Score is the average of **s** over all points. Its range is $[-1, +1]$:

- Close to +1 → points are well matched to their own cluster and far from others (good clustering).
- Close to 0 → points lie on the boundary between clusters.
- Negative → points may have been assigned to the wrong cluster.

In this experiment the highest Silhouette Score (0.5547) was obtained at **K** = 5, confirming the choice made by the Elbow Method.

6. Why is K-Means considered an unsupervised machine learning algorithm?

K-Means is unsupervised because it does not require any labeled target variable. The algorithm discovers structure in the data solely by measuring similarity (distance) between data points. There is no “correct” answer provided during training; the

algorithm itself decides which points belong together. This is in contrast to supervised algorithms (classification, regression) that learn a mapping from input features to a known output label.

7. Mention any four real-world applications of customer segmentation.

1. **Targeted Marketing Campaigns** – Different promotional offers are designed for high-spending vs. budget-conscious customers.
2. **Product Recommendation Systems** – E-commerce platforms recommend products based on the segment a customer belongs to.
3. **Customer Retention & Churn Prevention** – High-value customers at risk of leaving can be given special attention and loyalty benefits.
4. **Pricing Strategy & Inventory Planning** – Retailers stock and price products according to the dominant segments in each region or store.

Other applications include personalized banking services, insurance risk grouping, and content personalization on streaming platforms.

8. What are the limitations of the K-Means algorithm?

- The number of clusters K must be specified in advance.
- Sensitive to the initial placement of centroids (can converge to local minima).
- Assumes clusters are spherical and of similar size; performs poorly on elongated or irregularly shaped clusters.
- Sensitive to outliers because centroids are computed as means.
- Works only with numerical data (categorical features need encoding).
- Does not guarantee a global optimum; results can vary across different runs.

9. How can businesses use customer segmentation to improve marketing and customer retention?

Businesses can use the discovered segments in several practical ways:

- **Personalized Marketing** – Send relevant offers, emails and advertisements to each segment instead of a one-size-fits-all campaign.
- **Resource Allocation** – Concentrate expensive acquisition and retention efforts on high-value segments (e.g., Cluster 1 – High Income High Spending).
- **Product Development** – Design new products or service tiers that match the needs and budgets of specific segments.
- **Loyalty Programs** – Create tiered loyalty schemes that reward the most profitable customers and encourage lower-spending segments to increase engagement.
- **Churn Reduction** – Identify segments showing declining activity early and intervene with win-back campaigns.

Overall, segmentation converts a heterogeneous customer base into manageable, actionable groups, leading to higher conversion rates, better customer lifetime value, and improved ROI on marketing spend.

10. Compare K-Means Clustering with Hierarchical Clustering based on their working principles and applications.

Aspect	K-Means Clustering	Hierarchical Clustering
Working Principle	Partition-based. Iteratively assigns points to the nearest centroid and updates centroids until convergence.	Builds a hierarchy of clusters either by successive merging (agglomerative) or successive splitting (divisive).
Number of Clusters	Must be specified in advance (K).	Can be decided after seeing the dendrogram; no need to pre-specify.
Computational Cost	Generally faster and scales better to large datasets ($O(n \cdot K \cdot i \cdot d)$).	More expensive, typically $O(n^3)$ or $O(n^2 \log n)$; less suitable for very large data.
Shape of Clusters	Assumes roughly spherical clusters of similar size.	Can discover clusters of arbitrary shape depending on the linkage criterion.
Output	Flat partition (each point belongs to exactly one cluster).	Tree-like dendrogram showing nested relationships.
Sensitivity	Sensitive to initial centroids and outliers.	Less sensitive to initialization; different linkage methods handle outliers differently.
Typical Applications	Customer segmentation, image compression, document clustering, large-scale marketing analytics.	Taxonomy creation, gene expression analysis, small-to-medium datasets where hierarchy is meaningful.

Conclusion: K-Means is preferred when the dataset is large and a quick flat partitioning is required (as in this customer segmentation experiment). Hierarchical clustering is preferred when the hierarchical relationships between groups are important or when the optimal number of clusters is unknown.